

디자인패턴 (+GoF)



잘 설계된 S/W란?

고객의 요청들

1. 제작 요청 : 이거 만들어주세요.
2. 기능 추가 요청 : 이거 기능 추가해주세요.
3. 변경 요청 : 이 부분 수정해주세요.

잘 설계된 S/W 조건

1. 고객 요구사항 대로 개발이 되었는가? ← 기본
2. 구조가 쉽게 파악이 되는가?
3. 기능 추가를 쉽게 할 수 있는가?
4. 변경이 편리한가?

고객의 요구사항

잠만 잘수있게 해주세요.

- 저는 잠에서 잘 안깨요.
- 나중에 수정사항은 전달드릴게요.

이렇게 만들어진 결과물

오른쪽 집은,
변경 요청에 대응이 어렵다.

- 구조 파악이 힘들다.
- 창문 교체 요청
→ 집 전체가 무너질 수 있음



수정이 편리한 S/W 결과물

조립식 건물

- 지붕 교체 가능
- 문 교체 가능
- 창문 교체 가능



하지만, 아무리 잘 만든 S/W 라도 ...

당연하게도

무리한 고객 변경 요구사항은 수용 불가

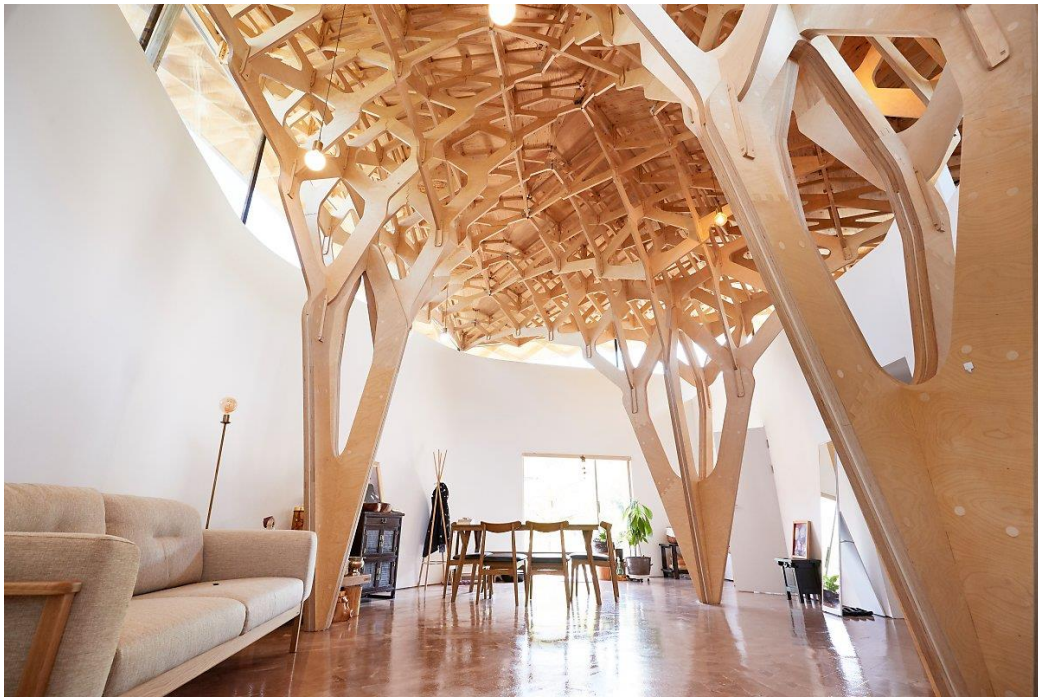


변경요청
수용불가

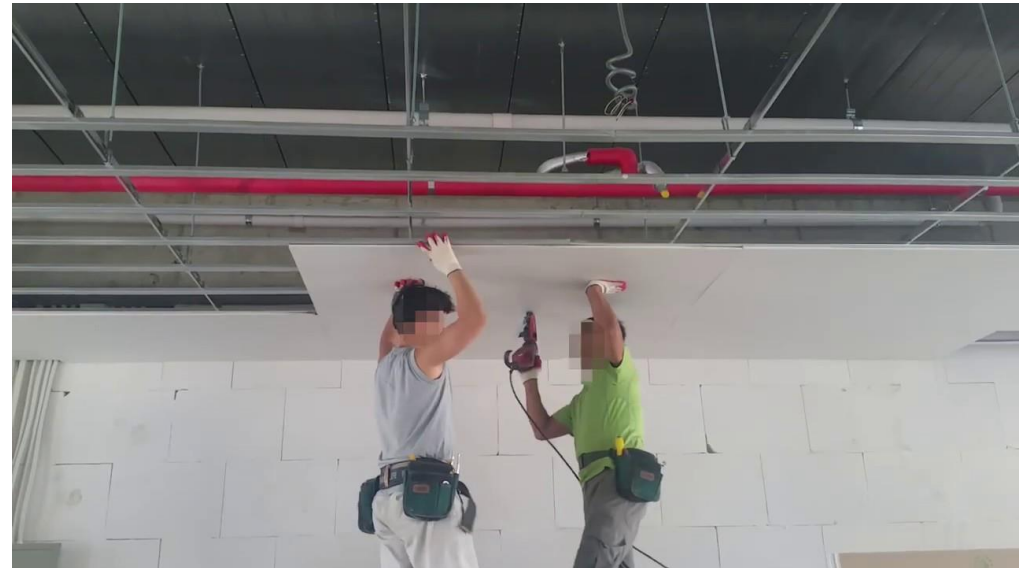


아름다운 구조? → 복잡하다.

구조가 단순해야, 유지보수가 된다.



유지보수가 힘든 복잡한 구조의 천장



오랜시간이 지나도, 유지보수가 용이한 구조

깔끔한 구조로 OOP 개발 하기 위한 훈련

1. 객체지향 원칙을 학습하는 방법

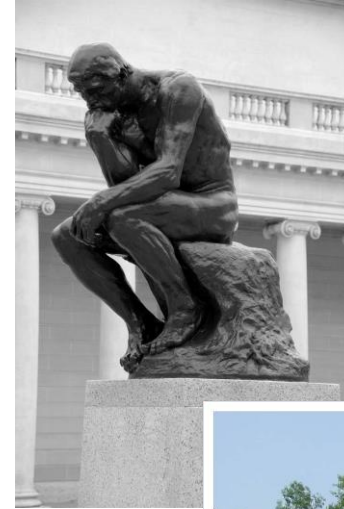
- SOLID 원칙

2. 깔끔한 코딩 뼈대를 학습하는 방법

- 디자인 패턴

3. 리팩토링

- 깔끔한 구조로, 지속적인 개선하는 훈련



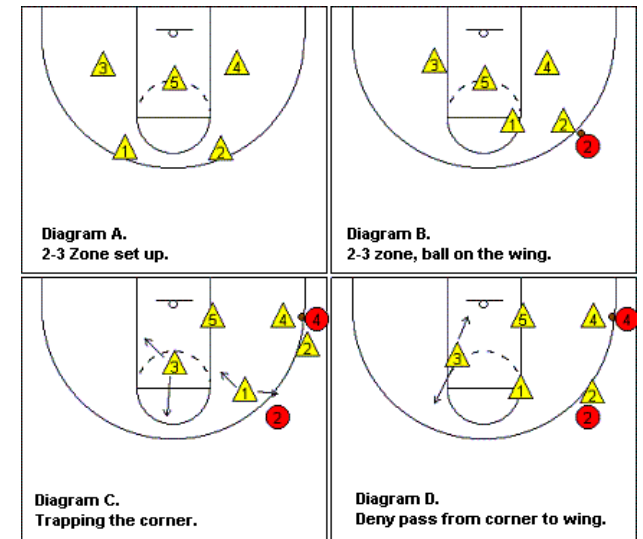
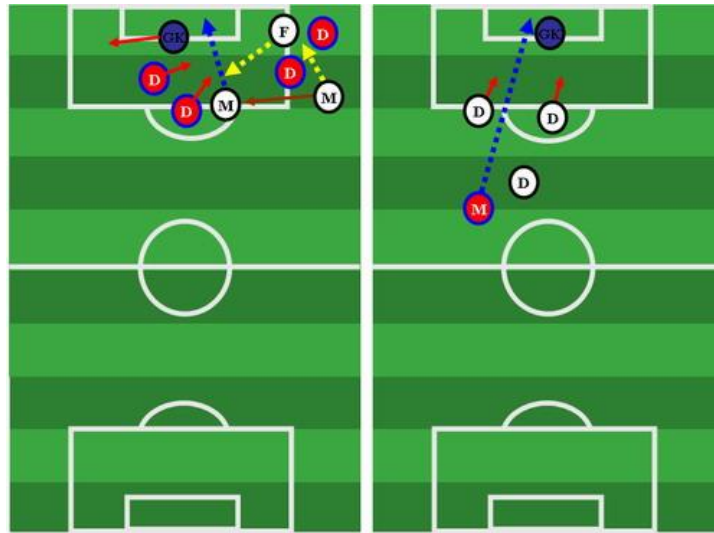
GoF 디자인패턴 소개

Gang of Four 가 만든, 소스코드 패턴



디자인 패턴이란?

자주 사용되는 설계를 일반화하여,
패턴으로 정리한 것



디자인 패턴의 장단점

장점

- 개발자간 의사소통
- S/W 구조 파악이 쉽다.
- 설계 변경 요청에 대해 유연한 대처 가능

디자인 패턴의 장단점

단점

- 개발에 참여하는 팀원 전체가 학습되지 않으면, 유지보수가 더 어려워진다.
- 구현 난이도가 올라간다.

GoF 디자인 패턴

소프트웨어 설계에 공통된 문제에 대한 표준 해법

- 네 명의 학자가 쓴 책이기에 Gang of Four, GoF 디자인패턴으로 불리운다.

GoF 디자인패턴 종류

생성, 구조, 동작 패턴으로 3개 그룹으로,
총 23개 디자인 패턴

- 생성 : Factory Method / Abstract Factory / Builder / Prototype / Singleton
- 구조 : Adapter / Bridge / Composite / Decorator / Facade / Flyweight / Proxy
- 동작 : chain-of-responsibility / command / interpreter / iterator / mediator / memento / observer / state / strategy / template method / visitor

실무에 자주 사용되는 패턴 표기

Dependency Injection (DI)

디자인패턴을 시작하기 전, 준비사항

Dependency Injection (DI)

객체 생성 방법 1 : Class 내부에서 객체 생성

- Life Cycle 모두 관리
- Tight Coupling

객체 생성 방법 2 : 밖에서 만든 객체를 넣어주기 ← DI

- 주입되면 사용만 함
- Loose Coupling

DI 예시

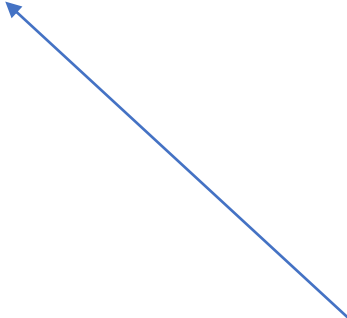
밖에서 만들어서
넣어주면 끝

```
public class Main {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
  
        //단순하게 만든것  
        BTS a = new BTS();  
  
        //만들어서 넣어주는 것 Dependency Injection - DI  
        abc(new BTS());  
    }  
  
    //v는 이미 만들어진것이고 그냥 난 사용만한다.  
    static void abc(BTS v) {  
  
    }  
}  
  
class BTS {  
  
}
```

Factory Method Pattern

객체를 생성해주는 공장

Factory Method 라고해서
Method 임을 나타내지 않는다. (함정)

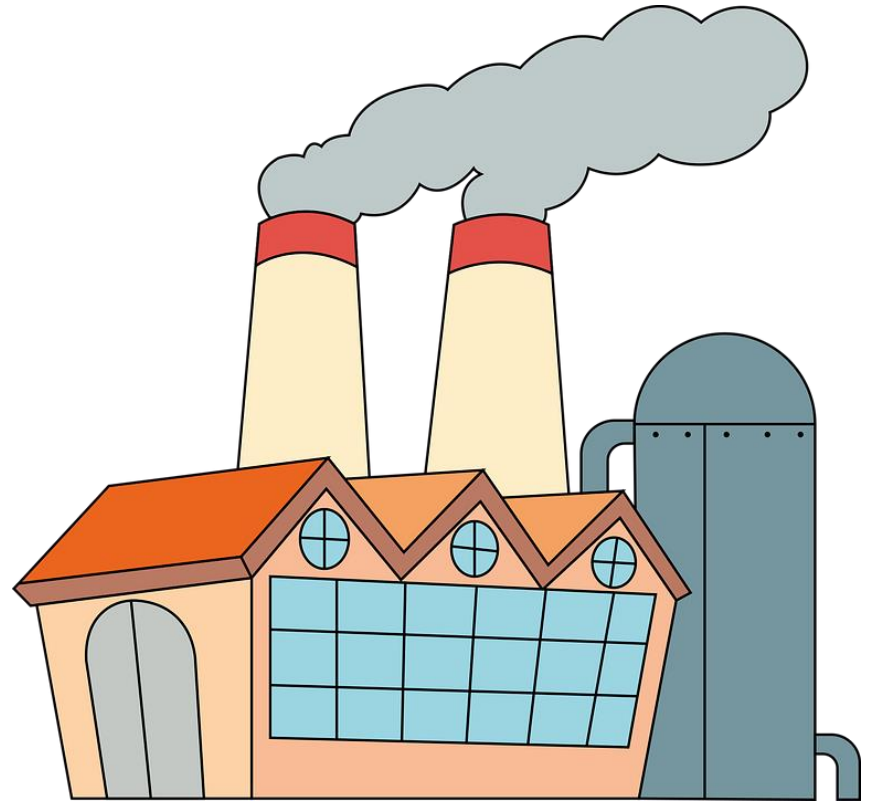


Simple Factory 형태

객체를 생성해주는 곳이 한곳이다.

- 한 곳에서 생성, 관리 가능

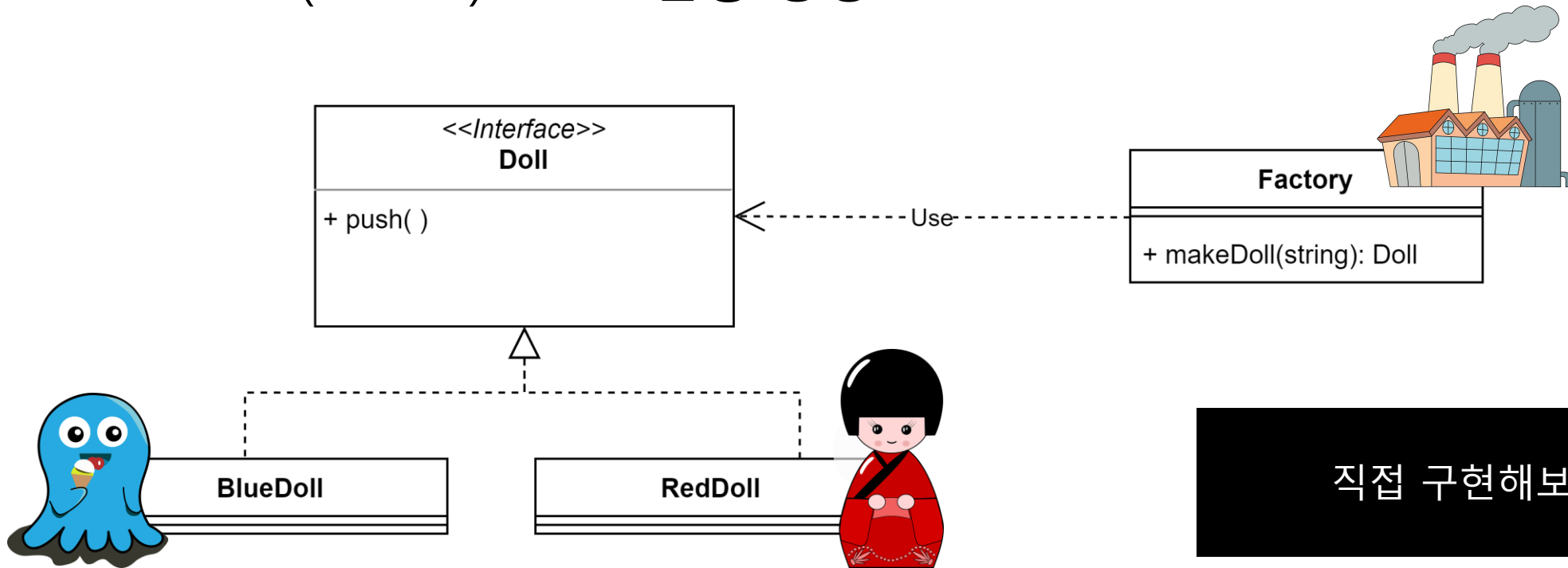
생성코드를 Client에 공개하지 않는다.



UML 분석 1 (Factory 형태)

makeDoll(string)

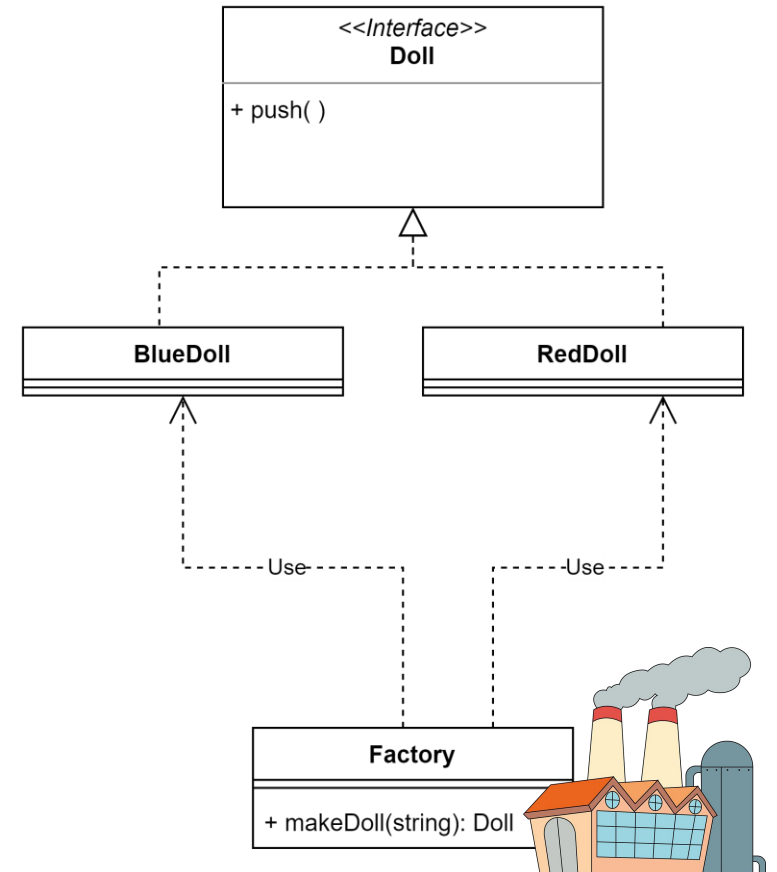
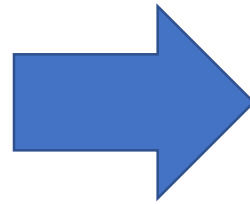
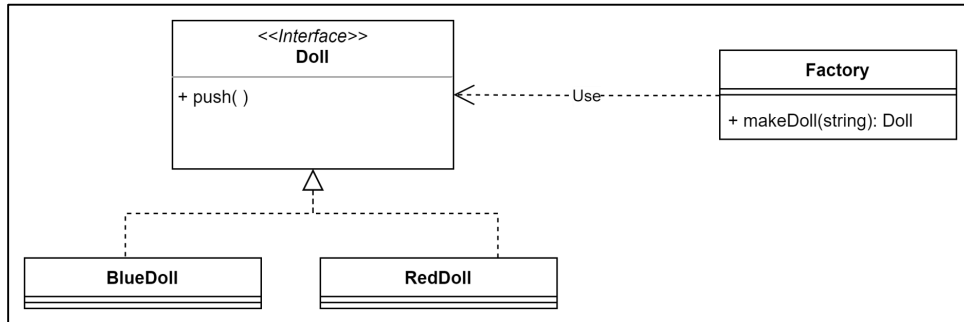
- makeDoll("RED") : Red 인형 생성
- makeDoll("BLUE") : Blue 인형 생성



직접 구현해보자

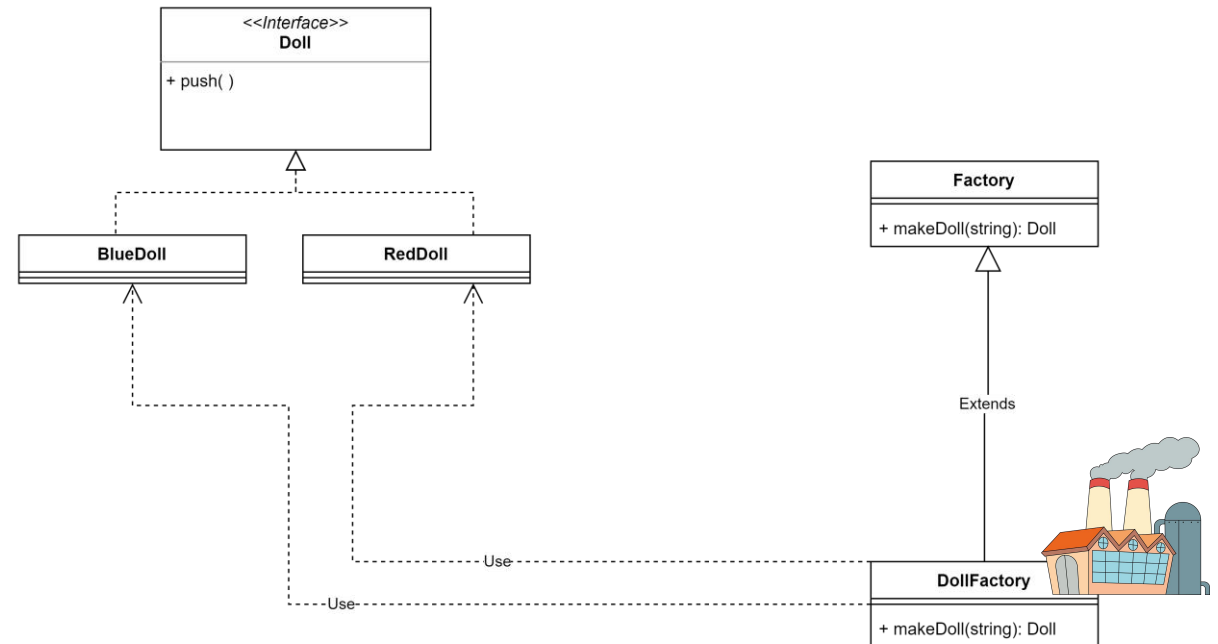
UML 분석 2 (Factory 형태)

더 정확한 UML 형태



Factory Method Pattern

여러개의 Factory 생성 가능



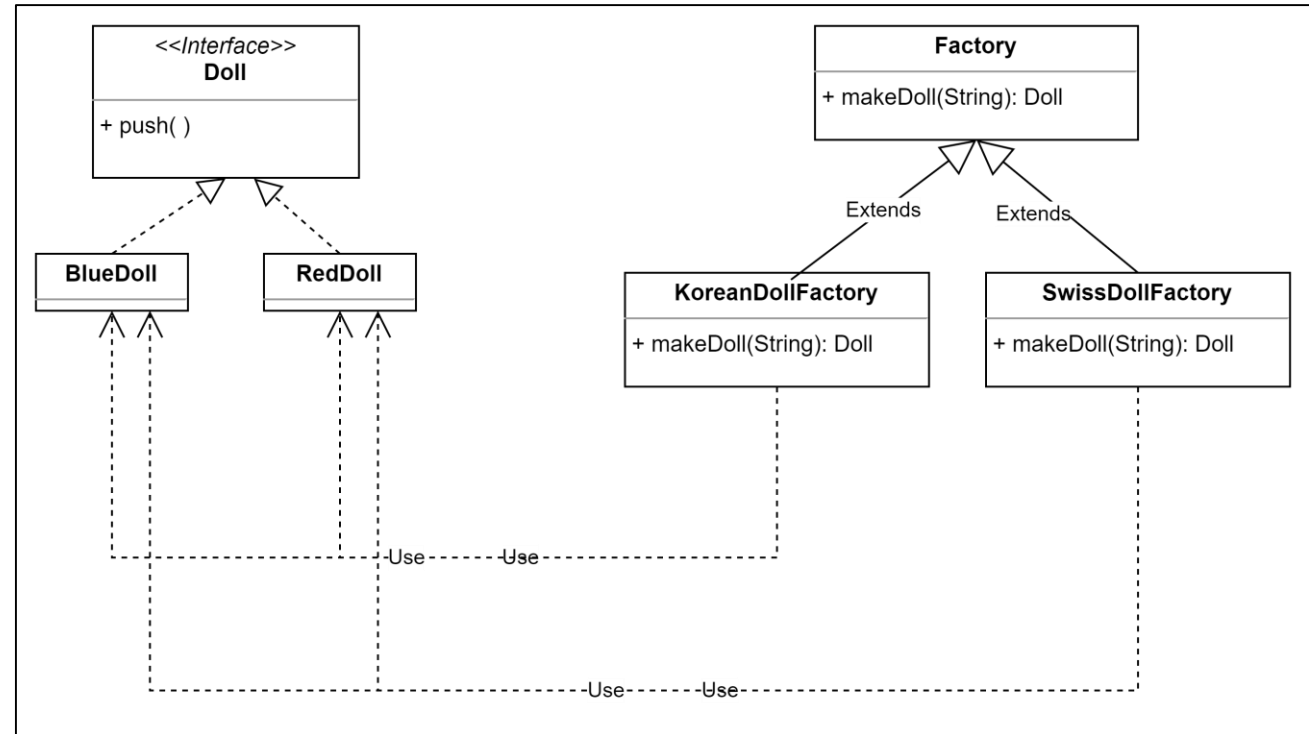
[도전] Factory Method Pattern 구현

KoreanDollFactory

- 한국어로 된 음성이 나온다.
 - BlueDoll : "파랑파랑"
 - RedDoll : "레드레드"

SwissDollFactory

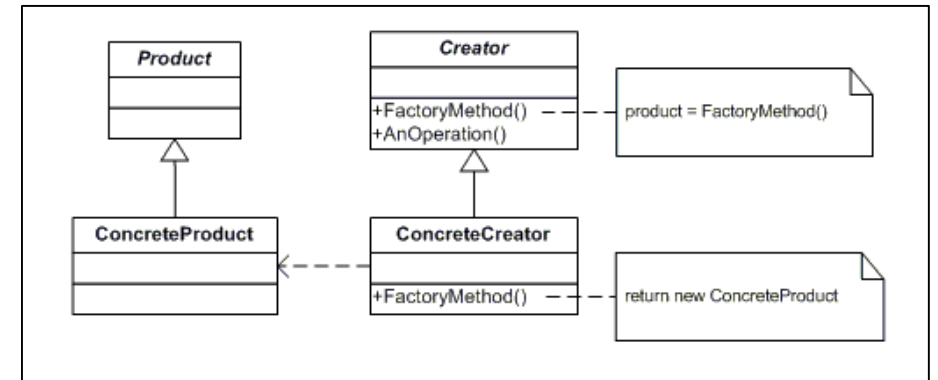
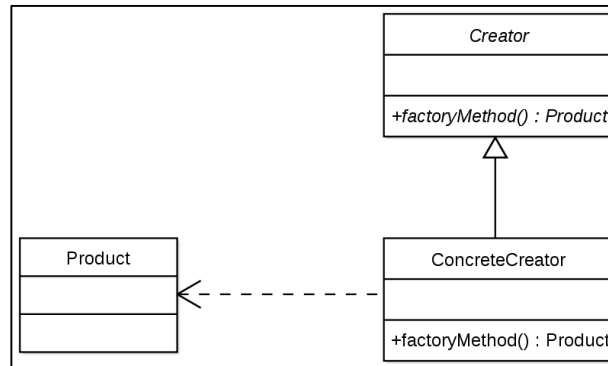
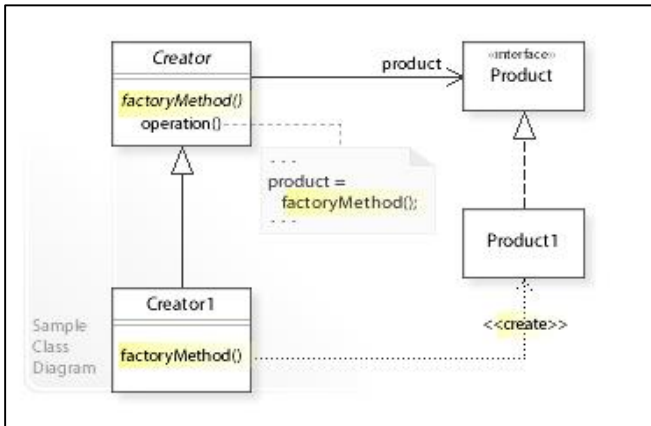
- 독일어로 된 음성이 나온다.
 - BlueDoll : "BuruGten"
 - RedDoll : "RetGten"



UML 표현 예시

Creator : Factory

ConcreteCreator : 구체(구상)적인 Factory



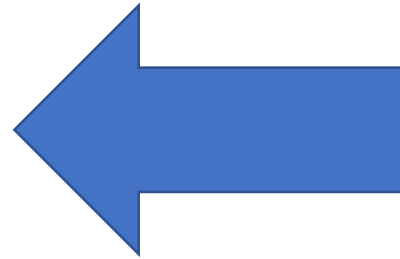
Builder Pattern (+ Effective Java)

생성을 더 간단하게.

두 가지 Builder Pattern

Effective Java의 Builder Pattern

- 생성자 인자가 많을때 사용한다.
- 메서드 체인



GoF 의 Builder Pattern

- 생성 절차가 복잡할 때 쓴다.

헤어컷

Client 구현

- Style명 : "군인컷"
 - 앞머리 : 2cm
 - 뒷머리 : 1cm
 - 옆머리 : 1cm
- Style명 : "아나운서컷"
 - 앞머리 : 6cm
 - 뒷머리 : 3cm
 - 옆머리 : 7cm



일반적인 객체 생성 방법

평범한 객체 생성과 setter 제작

```
public static void main(String[] args) {  
    Cut gunCut = new Cut("군인컷", 2, 1, 1);  
}
```

각 Argument가 무슨 뜻인지,
이 코드만으로는 알 수 없음



```
public static void main(String[] args) {
```

```
    Cut gunCut = new Cut();  
    gunCut.setStyle("군인컷");  
    gunCut.setFront(2);  
    gunCut.setBack(1);  
    gunCut.setSide(1);
```

```
    Cut anaCut = new Cut();  
    anaCut.setStyle("아나운서컷");  
    anaCut.setFront(6);  
    anaCut.setBack(3);  
    anaCut.setSide(7);
```

```
}
```

Cut
+ style: string + front: int + back: int + side: int
+ setStyle(string) + setFront(int) + setBack(int) + setSide(int)

메서드 체이닝

return 참조를 통한 코드 개선

```
public static void main(String[] args) {
```

```
    Cut gunCut = new Cut();  
    gunCut.setStyle("군인컷");  
    gunCut.setFront(2);  
    gunCut.setBack(1);  
    gunCut.setSide(1);
```

```
    Cut anaCut = new Cut();  
    anaCut.setStyle("아나운서컷");  
    anaCut.setFront(6);  
    anaCut.setBack(3);  
    anaCut.setSide(7);
```

```
}
```



```
public static void main(String[] args) {
```

```
    Cut gunCut = new Cut();  
    gunCut = gunCut.setStyle("군인컷").setFront(2).setBack(1).setSide(1);
```

```
    Cut anaCut = new Cut();  
    anaCut = anaCut.setStyle("아나운서컷").setFront(6).setBack(3).setSide(7);
```

```
}
```



```
public static void main(String[] args) {
```

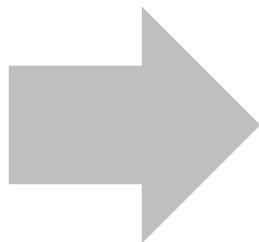
```
    Cut gunCut = new Cut().setStyle("군인컷").setFront(2).setBack(1).setSide(1);  
    Cut anaCut = new Cut().setStyle("아나운서컷").setFront(6).setBack(3).setSide(7);
```

```
}
```

코드 정리 결과

중복 요소 제거

```
public static void main(String[] args) {  
  
    Cut gunCut = new Cut();  
    gunCut.setStyle("군인컷");  
    gunCut.setFront(2);  
    gunCut.setBack(1);  
    gunCut.setSide(1);  
  
    Cut anaCut = new Cut();  
    anaCut.setStyle("아나운서컷");  
    anaCut.setFront(6);  
    anaCut.setBack(3);  
    anaCut.setSide(7);  
  
}
```



```
public static void main(String[] args) {  
  
    Cut gunCut = new Cut()  
        .setStyle("군인컷")  
        .setFront(2)  
        .setBack(1)  
        .setSide(1);  
  
    Cut anaCut = new Cut()  
        .setStyle("아나운서컷")  
        .setFront(6)  
        .setBack(3)  
        .setSide(7);  
  
}
```

Builder() 메서드

메서드 체인

- 이로 얻는 장점은 “가독성” 이다.

```
public static void main(String[] args) {  
    Person per = Person.Builder()  
        .setAge(10)  
        .setHeight(100)  
        .setIq(100);  
}
```

```
class Person {  
    private int age;  
    private int height;  
    private int weight;  
    private int iq;  
    private String mbti;  
  
    public Person setAge(int age) {  
        this.age = age;  
        return this;  
    }  
  
    public Person setHeight(int height) {  
        this.height = height;  
        return this;  
    }  
  
    public Person setWeight(int weight) {  
        this.weight = weight;  
        return this;  
    }  
  
    public Person setIq(int iq) {  
        this.iq = iq;  
        return this;  
    }  
  
    public Person setMbti(String mbti) {  
        this.mbti = mbti;  
        return this;  
    }  
  
    static Person Builder() {  
        return new Person();  
    }  
}
```

Singleton Pattern

단 하나의 Instance만을 사용한다.

싱글톤이란

객체가 생성되는 최대 개수로 생성을 제한하는 방법

- ex) 아래 소스코드 예제는 하나만 허용했을 때 상황

```
public static void main(String[] args) {  
  
    Wife jihyo = new Wife();  
    jihyo.date();  
  
}
```

문제없는 소스코드



```
public static void main(String[] args) {  
  
    Wife jihyo = new Wife();  
    jihyo.date();  
  
    Wife boyoung = new Wife();  
    boyoung.date();  
  
}
```

Client Level의 옳지 못한 구현



static 메서드 이해하기

Instance를 생성하지 않고, 접근 가능함

```
class BTS {  
    static void run() {  
        System.out.println("WOW");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        BTS.run();  
    }  
}
```

Singleton Pattern

1. 생성자를 `private`로 만들어,
Client에서 생성하지 못하도록 막는다.
2. `getInstance( )` 메서드를 통해서만
해당 객체를 생성한다.

Singleton	
-	<u>singleton : Singleton</u>
-	Singleton()
+	<u>getInstance() : Singleton</u>

Singleton 소스코드

Wife 이름을 확인하여,
같은 사람임을 확인할 수 있음

```
public static void main(String[] args) {  
  
    Wife jihyo = Wife.getWife();  
    jihyo.setName("JIHYO");  
  
    Wife boyoung = Wife.getWife();  
    System.out.println(boyoung.getName());  
}
```

```
public class Wife {  
  
    String name;  
    static Wife mine = new Wife();  
  
    private Wife() {  
    }  
  
    static Wife getWife() {  
        return mine;  
    }  
  
    void date() {  
        System.out.println("HAHA");  
    }  
  
    void setName(String name) {  
        this.name = name;  
    }  
  
    String getName() {  
        return name;  
    }  
}
```

문제점

프로그램 시작되자마자
Wife가 바로 생성됨

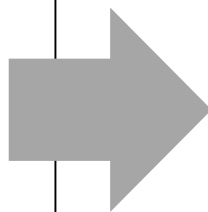
```
public static void main(String[] args) {  
  
    Wife jihyo = Wife.getWife();  
    jihyo.setName("JIHYO");  
  
    Wife boyoung = Wife.getWife();  
    System.out.println(boyoung.getName());  
}
```

```
public class Wife {  
  
    String name;  
    static Wife mine = new Wife();  
  
    private Wife() {  
    }  
  
    static Wife getWife() {  
        return mine;  
    }  
  
    void date() {  
        System.out.println("HAHA");  
    }  
  
    void setName(String name) {  
        this.name = name;  
    }  
  
    String getName() {  
        return name;  
    }  
}
```

개선

필요시
객체 생성

```
public class Wife {  
  
    String name;  
    static Wife mine = new Wife();  
  
    private Wife() {  
    }  
  
    static Wife getWife() {  
        return mine;  
    }  
  
    void date() {  
        System.out.println("HAHA");  
    }  
  
    void setName(String name) {  
        this.name = name;  
    }  
  
    String getName() {  
        return name;  
    }  
}
```



```
String name;  
static Wife mine;  
  
private Wife() {  
}  
  
static Wife getWife() {  
    if (mine == null) mine = new Wife();  
    return mine;  
}
```

정리, 기본 싱글톤

직접 구현해보기

```
class KFC {  
    private KFC() {}  
    public static KFC getInstance() {  
        if (handle == null) {  
            handle = new KFC();  
        }  
        return handle;  
    }  
  
    private static KFC handle;  
  
    public void hi() {  
        System.out.println("HI I'm KFC");  
    }  
}
```

싱글톤 개발하기

Logger와 바람둥이 프로그램 제작



[도전] Logger 제작하기

로그라이터 클래스 제작하기

- BufferedWriter 로 파일 입출력
- Write 메서드 제공

싱글톤으로 구현하기

```
public class Main {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Logger log1 = new Logger();  
        log1.write("HI");  
  
        Logger log2 = new Logger(); //문제발생!!  
        log2.write("HI");  
    }  
}  
  
class Logger {  
  
    Logger() {  
        //파일Open("ABC.txt");  
    }  
  
    public void write(String str) {  
        //파일에 str 쓰기  
    }  
}
```

[도전] 못된 바람둥이

싱글톤으로, 애인 생성 3명 까지 허용

- 한번 더 요청시, 첫번째 부인으로 지정하기

```
public class Main {  
    public static void main(String[] args) {  
  
        Wife w1 = Wife.getWife("JIHYO");  
        Wife w2 = Wife.getWife("MEHEE");  
        Wife w3 = Wife.getWife("BOYOUNG");  
        Wife w4 = Wife.getWife("WHASO");  
  
        //단 세명의 와이프만 생성되도록 해야함  
        System.out.println(w1.getName()); //JIHYO  
        System.out.println(w2.getName()); //MEHEE  
        System.out.println(w3.getName()); //BOYONG  
        System.out.println(w4.getName()); //JIHYO  
    }  
}
```



Pattern 을 사용한 개발 PJT 1

Singleton + Builder + Factory Method Pattern



도전과제

1. Factory Method Pattern (또는 Simple Factory)
2. 메서드체인 형태의 Builder
3. Singleton 을 모두 사용하여 개발한다.
 - Factory + Singleton = 자주 사용 됨

GoldFactory 만들기

만들어내는 제품

- GoldBar



이름, 가로, 세로, 높이 사이즈를 지정할 수 있다.

각 제품들은 `showInfo()` 메서드를 갖는다.

- 제품의 이름, 사이즈, 가격이 출력된다.
- 가격 = 가로 x 세로 x 높이 x 가치 (임의지정)

커맨드 패턴

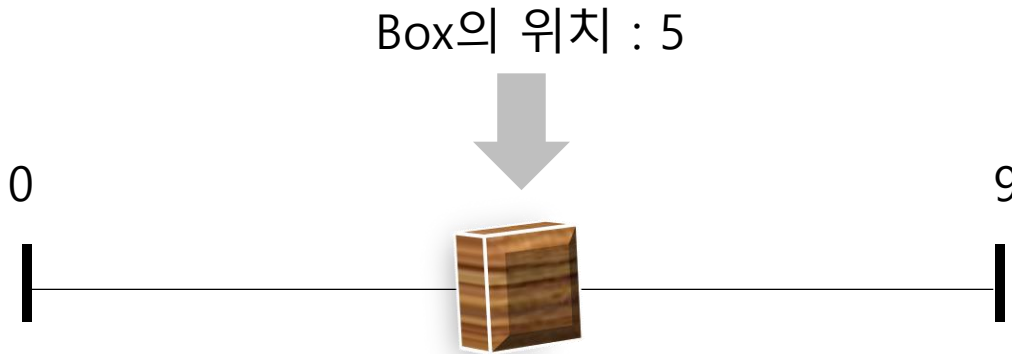
명령대로 수행해주는 프로그램 제작시 사용

C-업 락 히트	
C-업 락 크랙	
C-업 락 스윙	
호핑	
엑스애어리얼	
엑스애어리얼 - 더블	
비트윈보	
C-클램프	
C-클램프 - 스페셜	

박스 준비

박스 클래스를 하나 준비한다.

- 최소값 : 0
- 최댓값 : 10
- 초기 박스의 위치 : 5



```
class Box
{
    private int x = 5;

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }
}
```

명령어 기반 프로그램

명령을 받고 수행하는 프로그램

- LEFT 명령어
 - Box가 왼쪽으로 이동
- RIGHT 명령어
 - Box가 오른쪽으로 이동
- SHOW 명령어
 - Box가 화면에 출력되는 프로그램

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    Box box = new Box();  
  
    while(true) {  
        System.out.print("명령어 입력 : ");  
        String cmd = sc.next();  
  
        if (cmd.equals("left")) left(box);  
        if (cmd.equals("right")) right(box);  
  
        show(box);  
    }  
}
```

함수가 아닌, Command 객체로 만들기

명령어를 추가하기 위한 변경

1. 메서드 하나 추가
2. Command 수행 소스코드 수정

메서드 대신
Command 객체로 만들어보자.

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Box box = new Box();

    while(true) {
        System.out.print("명령어 입력 : ");
        String cmd = sc.next();

        if (cmd.equals("left")) left(box);
        if (cmd.equals("right")) right(box);

        show(box);
    }
}
```

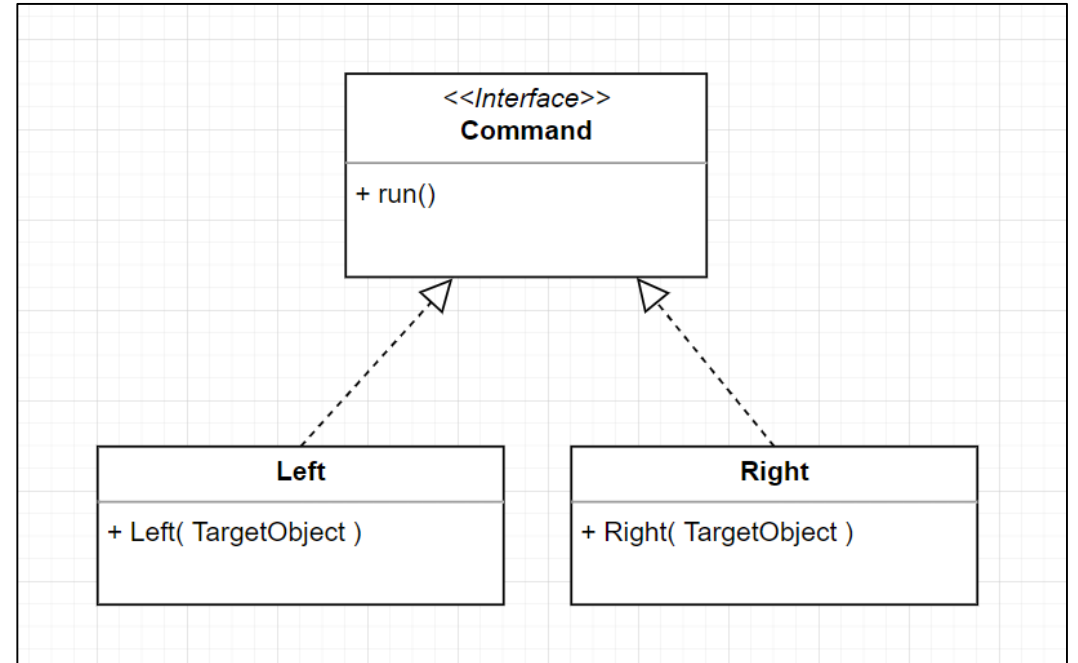
```
static void left(Box box) {
    box.setX(box.getX() - 1);
}

static void right(Box box) {
    box.setX(box.getX() + 1);
}

static void show(Box box) {
    for (int i = 0; i < 10; i++) {
        if (box.getX() == i) {
            System.out.print("#");
        }
        else {
            System.out.print("_");
        }
    }
    System.out.println();
}
```

[도전] Command 객체 만들기

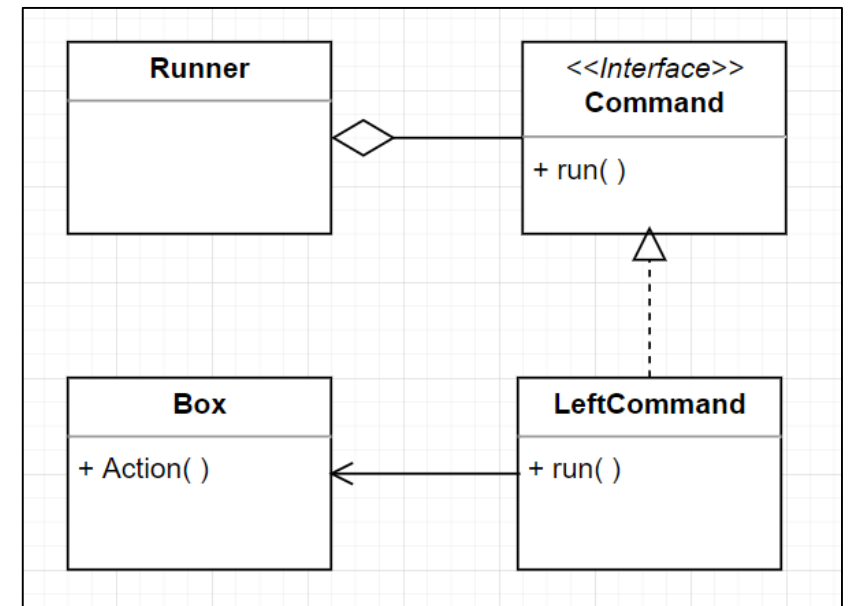
1. Left, Right 객체를 만든다.
2. 기존 소스코드를 해당 객체로 치환한다.



Command Pattern 구현하기

Command를 함수로 구현하지 않고,
객체로 만들어서 관리하는 패턴

1. Command를 객체화
2. Runner 에서 Command 관리
 - Command 로깅
 - Command 목록 자동화
 - Command 취소 기능 제작 가능



Client에서 동작 수행은 Runner를 통해 이뤄진다.

[도전] 스위치 제작하기

Runner의 명령어

- flipUp
- flipDown

Command 객체

- TurnOn
- TurnOff



Strategy 패턴

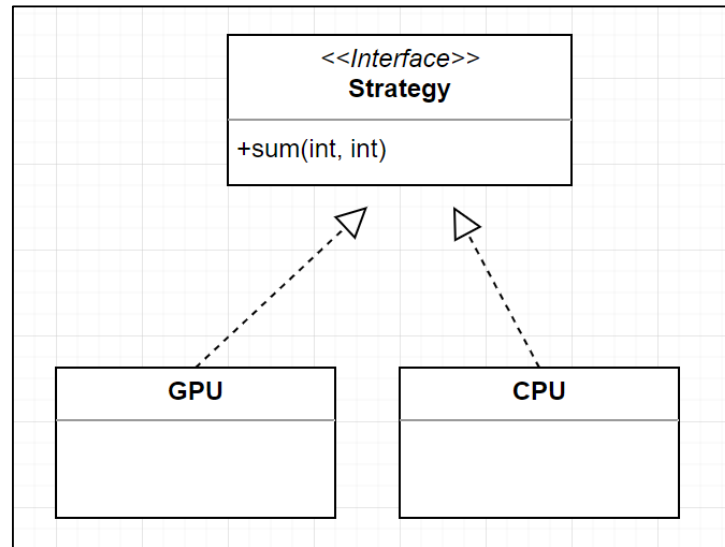
여러 알고리즘 중 하나를 선택할 수 있게 만드는 패턴



GPU Class, CPU Class

덧셈기 제작하기

- GPU Module : GPU를 사용한 연산 후, 결과 리턴해주는 객체
- CPU Module : CPU를 사용한 연산 후, 결과를 리턴해주는 객체



[도전] 동적 선택, 프로그램 제작

질문 1 예시

- 연산 방법을 선택하시오.
 1. GPU를 사용한 연산
 2. CPU를 사용한 연산

질문 2 예시

- 수 2개를 입력하시오 : 3 5

출력결과 예시

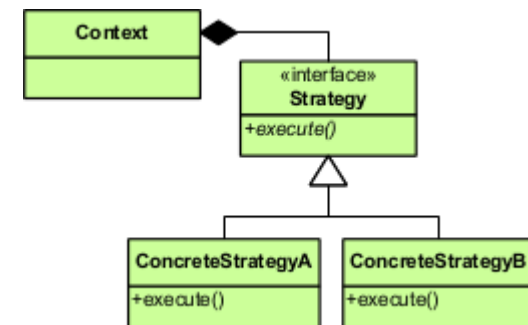
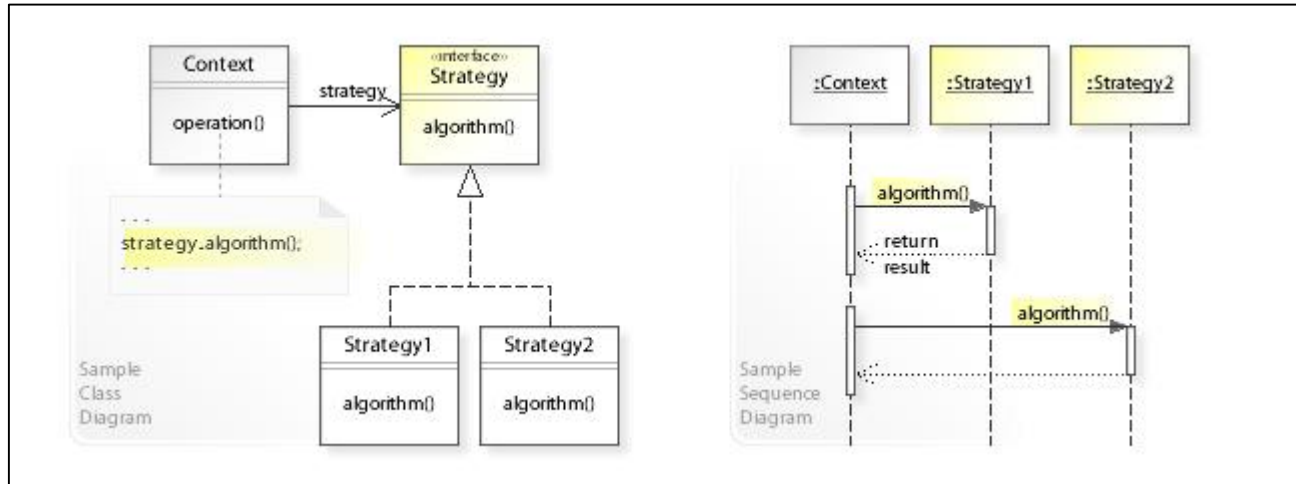
- 8

전략패턴 UML

Command Pattern 과 유사하다.

알고리즘을 Runtime 중에 선택할 수 있는 패턴

- Interface로 하나 빼놓고, 선택 후 Run



전략패턴 코드

Manager는 Command Pattern의 Runner와 같은 역할

```
class Manager {  
    Strategy nowSt;  
    void selectStrategy(Strategy st) {  
        nowSt = st;  
    }  
  
    int sum(int a, int b) {  
        return nowSt.run(a, b);  
    }  
}
```

```
static public void main(String args[]) {  
  
    Manager man = new Manager();  
  
    Scanner sc = new Scanner(System.in);  
    int cmd;  
    int a, b;  
  
    System.out.print("연산방법 선택 (1GPU, 2CPU) : ");  
    cmd = sc.nextInt();  
  
    System.out.print("두 수 입력 : ");  
    a = sc.nextInt();  
    b = sc.nextInt();  
  
    if (cmd == 1) man.selectStrategy(new GPU());  
    if (cmd == 2) man.selectStrategy(new CPU());  
  
    System.out.println(man.sum(a, b) + " 입니다");  
}
```

Client에서 동작 수행은 Runner를 통해 이뤄진다.

Strategy vs Command Pattern

Command Pattern

- 명령어들을 수집하여 한꺼번에 Run 할 수 있음
- 명령어들을 모아 관리 가능

Strategy Pattern

- 알고리즘들을 모아두고,
여러 방법 중 한 방법을 선택해서 수행함

[도전] 복습, 전략패턴으로 다시 짜보기

질문 1 예시

- 연산 방법을 선택하시오.
 1. GPU를 사용한 연산
 2. CPU를 사용한 연산

질문 2 예시

- 수 2개를 입력하시오 : 3 5

출력결과 예시

- 8